QA/QC - Skill Assessment

This assessment will help us evaluate your ability to create test cases for software QA/QC, as well as identify areas for improvement. The scenarios presented will reflect real-world challenges you might encounter as a senior QA/QC tester. The goal is to assess your analytical thinking, attention to detail, and understanding of QA/QC principles in developing comprehensive test cases for various software applications.

Part 1 – Creating test cases

Background

You will receive an email giving you access to the BIS Assessment Portal, a test environment for our skill assessments. The email will provide you with the instructions on how to setup your account. Please follow those instructions and get your account activated.

Once your account is setup, we'll ask you to watch this video:

Adding documents to a Digital Folders

In this video, one of our team members will run you through a settings page in one of the modules in our system – Folders. Following this video, we'll ask you to complete the following task:

The Challenge

Based on the information provided in the video, create a list of test cases that can be used to verify the functionality of the settings page. Your test cases should be used for regression testing whenever updates are made to this page, ensuring that existing functionalities remain unaffected by any future changes.

The tests case can be created on a Word Document or Excel spreadsheet. Please upload the completed test cases to the link provided in the initial email.

Part 2 – Identifying Opportunities for Improvement

Background

After completing Part 1, you will continue to work within the BIS Assessment Portal. You will use the same settings page in the "Folders" module from the video tutorial.

The Challenge

Your task is to carefully review the functionality of the settings on the 'Add Document' page and identify any potential bugs or areas for improvement. Provide a detailed list of the issues you find, including their potential impact on the user experience or the system's performance. For each bug or improvement suggestion, describe why it is important and propose potential solutions or optimizations.

Additionally, consider any enhancements that could improve the usability or functionality of the settings page, and include these in your list of suggested improvements.

Part 3 – Automation Testing

Scenario

You are automating a document management feature on a web-based LMS (Learning Management System). The application allows administrators to upload documents into a folder. The UI has dynamic behavior: some form sections only appear after specific checkboxes are clicked, and the Submit button's surrounding structure may change depending on which optional sections are expanded. After clicking Submit, the app makes an API call to save the document — the document should then appear in the folder's document list.

Your Task

Using **Playwright (TypeScript)**, write a complete, production-ready automation test that does **all** of the following:

1. Navigate to the Add Document Form

Navigate to the Folders section, open a specific folder, and click the **Add (+)** button to open the **Add Document** form.

2. Fill in the Form with Dynamic Behavior

Fill in the form with the following actions **in order**:

1. Enter a **Document Title**
2. Click the "**+ Upload**" button and attach a local test file
3. Check the **"Include Acceptance Terms/Conditions Statement"** checkbox
    a. Assert that the Terms section fields are now **visible** in the DOM after checking (they are conditionally rendered)
4. Check the "**Include Notification Alert**" checkbox
    a. Assert that the Notification editor section is now **visible**

3. Intercept the Network Request Before Asserting the UI

Before clicking Submit:

1. Set up a **Page.waitForResponse()** listener that waits for the API call triggered by form submission.

Then:

1. **Click the Submit button**
2. **Await the network response and assert:**
    a. **The response status is 200**
    b. ***(Bonus)* Parse the response body and assert it contains a success indicator (e.g. "Success": true or no error message).**

4. Verify the Document Was Added Successfully

After the form submission redirects back to the folder's document page:

1. Assert that the page URL has navigated back to the folder documents view.
2. Assert that the newly added document's title is **visible on the page**

3. Assert that a **document card or list item** containing the title also contains either an **Edit or Archive** button — confirming the document is fully rendered as a document entity (not just a text match)

Challenge

Submit a **.spec.ts** file. Include brief inline comments explaining non-obvious choices (e.g. why you used Promise.all, why you scoped your final assertion the way you did).

Thank you for taking part in this assessment!